

Project Report

Project ID: BSDA2001P

Project Name: Introduction to DL and GenAI Project

Name: Vignesh N

Roll No: 23F2000391

Abstract – The main objective of this project is to build AI systems that can rank probable options for each question (A-E) and then predict the top 3 likely answers. The models are evaluated using MAP@3 (Mean Average Precision at 3), while other metrics such as Top-1 Accuracy, Top-3 Accuracy and Macro-F1 Score are used while training the models. Overall, five approaches were used. The scratch model created was an attention-based BiLSTM model which performed the best with an MAP@3 score of 0.7643. The pre-trained models included a zero-shot classification using Qwen2.5-7B-Instruct and DeBERTa Transformer model from Hugging Face. For the models of choice, a pipeline involving TF-IDF and Logistic Regression was used and a RAG System using the Wikipedia dataset and a retrieval-based approach with the pre-processed train dataset as knowledge bases was implemented. TF-IDF provided a baseline and achieved 0.959 MAP@3 validation score. The Custom BiLSTM model and DeBERTa Transformer achieved perfect validation score of 1 after training on the dataset. The retrieval pipeline with train dataset also performed well achieving near-perfect validation MAP@3 (0.9975) while the zero-shot classification only achieved 0.87 Validation score. The key learnings from these experimentations were that the transformer models and the retrieval pipeline achieved the highest MAP@3 score by training on the train dataset which contained most of the questions that were present in the test dataset.

Key Words: BiLSTM, DeBERTa, MAP@3, RAG, Zero-Shot Classification, TF-IDF, Logistic Regression

I. Introduction

The project's objective is to design an AI-System approach to predict the top-3 most likely answer for a given question. Each question contains 5 options from A-E and the model is trained to predict top 3 choices. The models are evaluated on MAP@3 Score which rewards the model for placing the correct answer in higher position. The train dataset consists of 2000 rows of questions with 5 Options (A-E) each and the correct option for each question. Models were trained on them to predict a test dataset of 500 questions with 5 options each.

The project focused on experimenting with different DL Gen AI models and ranking the models based on their MAP@3 validation score and leaderboard score. The baseline threshold was to develop a model that crossed 0.73 MAP@3 score in the leaderboard. The models that were experimented are TF-IDF with Logistic Regression, custom BiLSTM with Attention Scores, DeBERTa Transformer, a zero-shot pipeline and a retrieval-based approach. The project also experimented with tools such as wandb (for Hyper Parameter Tuning with sweeps) and Hugging Face (for pre-trained models and deployment).

The rest of this report is organized as follows: **Section II** describes the dataset using the exploratory data analysis, and preprocessing techniques that were executed. **Section III** discusses the tokenization strategy that was used in the best model (BiLSTM) and why it was chosen and how it was implemented. **Section IV** discusses the 3 different types of models.

They include the Fine-Tuned Transformer, custom deep-learning, and model of choice (Retrieval-based and TF-IDF). **Section V** compares the performance of all 5 models using their validation metrics from wandb. **Section VI** summarizes the key findings, challenges faced while working on the project. It also discusses about the possible directions for future work, while **Section VII** provides the references used throughout the project.

II. Dataset & Preprocessing

The train and test dataset contains 2000 and 500 rows respectively of a question and 5 options (A-E) per row. Additionally, the correct option was present as a column in the train dataset. Models trained on the train dataset questions and predicted the top 3 likely correct options for each row in the test dataset.

During EDA, it was found that out of 2000 rows, only 1817 rows were unique in the train dataset. On further analysis, it was found out that all the questions had either a prefix or a suffix that was irrelevant to the question. When these were removed, there would only be 588 unique questions out of 2000. The prompts without prefixes and suffixes were classified into different categories of domains using a zero-shot classification.

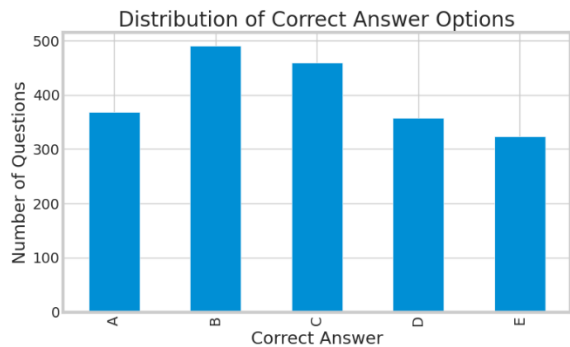


Figure 1: Class Distribution

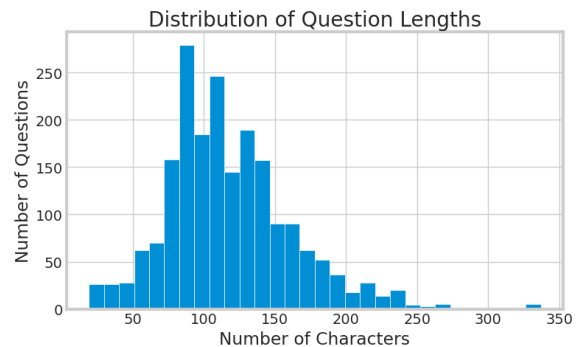


Figure 2: Character Distribution for Prompts

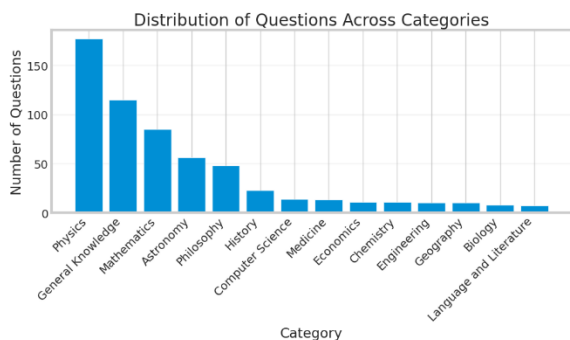


Figure 3: Category of Questions

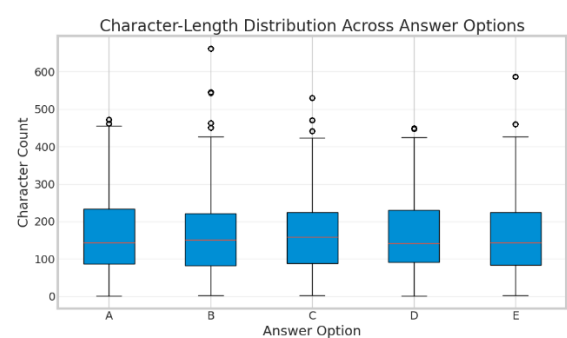


Figure 4: Character Distribution for Options

Figure_1 shows the distribution of correct answers which is not heavily imbalanced with Option B being the most frequent and Option E being the least frequent. Figure 2 describes the number of characters in the prompts while Figure 4 does the same for each option. This was helpful in selecting the max_length for tokenization. Figure 3, although might not fully accurate, provides a basic understanding of the domain of the questions.

Preprocessing techniques included converting values to strings, Unicode normalization, removing unnecessary whitespaces. The prompts were then cleaned by removing unnecessary suffixes and prefixes which may increase the number of tokens used and thus creating a new

column for clean_prompt. Some models such as BiLSTM preferred clean_prompt while models like TF-IDF preferred original prompts. No data augmentation was done since altering the questions or choices could change their meaning and hurt the models' performance.

III. Tokenization Strategy

The bert-base-uncased tokenizer was used for the final BiLSTM + Attention model. Although the tokenizer originates from the BERT architecture developed by Google, only its WordPiece tokenization mechanism was used. This tokenizer converts input text to lowercase and represents it using WordPiece subword tokens. These are then mapped to token IDs. Common words are represented as complete tokens, while uncommon or out-of-vocabulary words are split into smaller subword units.

This approach provides better coverage of uncommon words, and technical jargons without requiring the BiLSTM model to maintain an excessively large vocabulary. The tokenizer was especially effective with the dataset since most questions and options were similar and repetitive,

The Hugging Face AutoTokenizer interface was used to load the tokenizer. For each multiple-choice question, the question text and each of the five answer choices were tokenized as a question-option pair using max length of tokens as 256. The resulting token IDs were passed through a trainable embedding layer before being processed by the BiLSTM and attention mechanism.

IV. Modelling & Experimentation

A. Fine Tuned Transformer Model (DeBERTa-v3-base)

Original Research Paper: He, P., Gao, J., and Chen, W. (2023). "DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing." *International Conference on Learning Representations (ICLR)*. <https://doi.org/10.48550/arXiv.2111.09543>

Hugging Face Model Card: <https://huggingface.co/microsoft/deberta-v3-base>

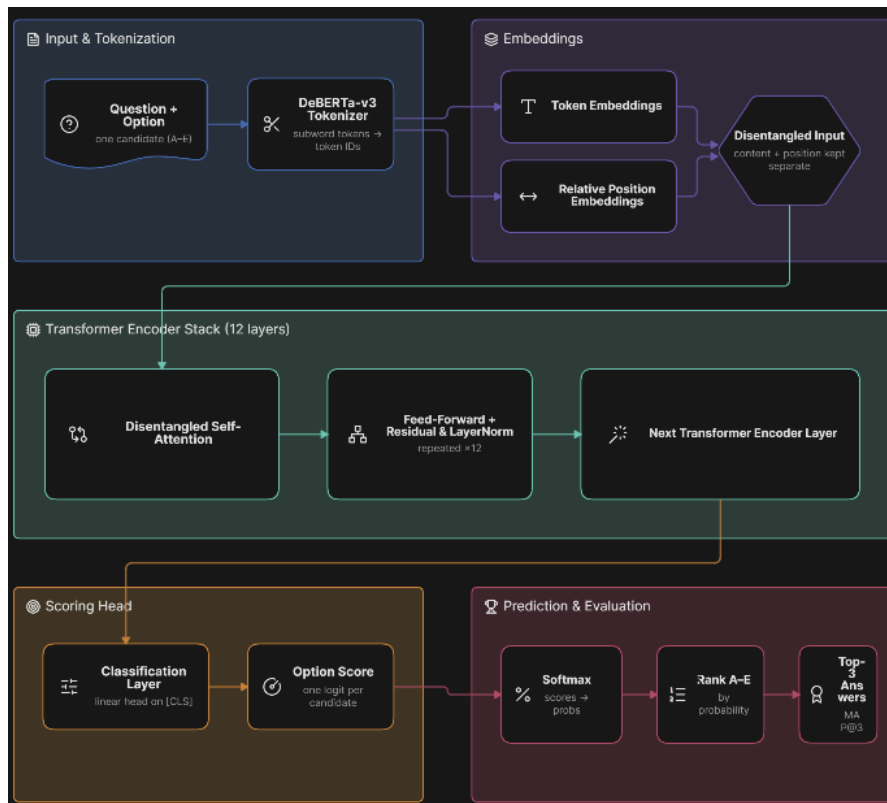


Figure 5: Arch. Diagram for DeBERTa

DeBERTa transformer model from Hugging Face is used as the fine-tuned transformer model to predict the top-3 options. DeBERTa extends the BERT architecture by introducing disentangled attention, where the content and positional information of tokens are represented separately. Unlike standard self-attention, where token representations combine content and position information directly, DeBERTa models these two components separately and calculates their interactions during attention computation. DeBERTa-v3 also uses an ELECTRA-style replaced token detection task, which improves the efficiency of pre-training by using replaced-token detection rather than relying only on masked language modeling. These improvements allow the model to learn strong contextual representations while maintaining the encoder-based architecture required for classification tasks.

Salient features of this project include the model's ability to consider information from both directions when representing tokens. The model's disentangled attention mechanism helps in representing the content and positional information separately. Its self-attention can help the model capture relationships between the question and its options. The model already has a large-scale corpora that it was trained on. These characteristics make DeBERTa particularly suitable for distinguishing between answer choices that may be semantically similar but differ in their relationship to the question.

The pretrained DeBERTa model was trained on the training dataset. Each question was paired with each of its 5 options. The model then generated a score of probability for each of these five choices. These scores were then ranked and the top-3 answers were chosen. Cross-Entropy Loss was used for optimization with AdamW as the optimizer. A learning-rate scheduler was used to adjust the learning rate during training. These helped the model achieve a MAP@3 score of 1.

B. Custom DL Model (BiLSTM + Attention)

For the scratch model, a bidirectional LSTM (BiLSTM) was designed. It also uses attention scores to improve predictive accuracy and suppress irrelevant noise.

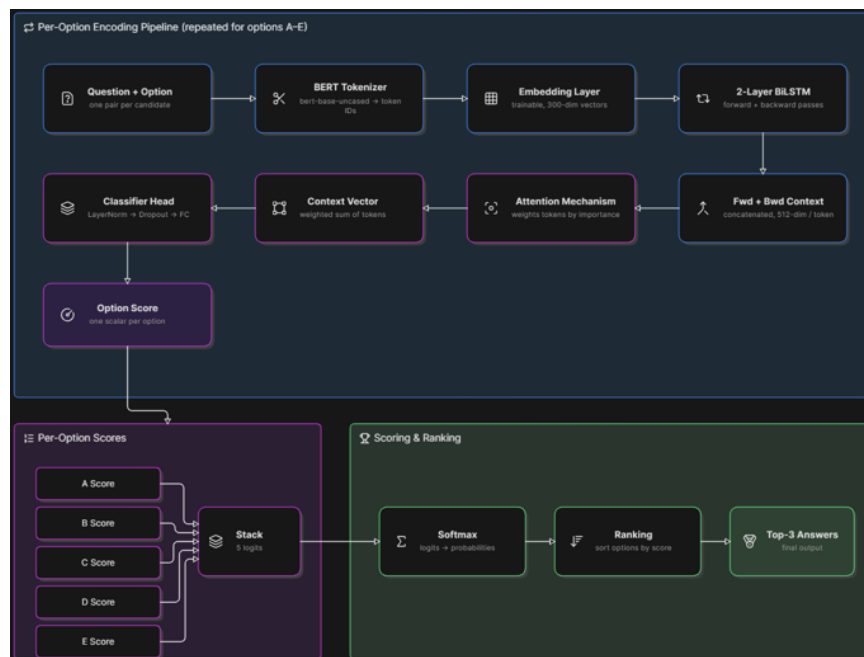


Figure 6: Arch. Diagram for BiLSTM Model

During implementation, for a row, the question and its 5 options were tokenized separately. The embedding layer then converts these token IDs into dense vectors. The forward and backward LSTM states allowed the model to capture information from both directions of the token sequence. Then different weights are assigned to the hidden states using the attention mechanism. This allows the model to give more importance to the more informative parts of the question-option sequence. The resulting context vector was normalized and passed through a feed-forward classifier to produce a score for the specified option. This process was repeated for all five options. The five scores were then ranked to obtain the final top-3 prediction.

The salient points of the custom architecture include the embedding layer which converts discrete token IDs into dense representations. The model captures sequential relationships in both forward and backward directions. The attention score helps the model to give greater importance to more information tokens. Dropout was used to reduce overfitting which was a major flaw for this project.

The model performed particularly well on the competition dataset because the training and test questions exhibited significant similarity. The model was therefore able to learn useful patterns from the training examples and achieve a validation MAP@3 of 1.

C. Model of Choice (Retrieval-Based Approach)

The third approach uses a retrieval-based system to identify previously seen or semantically similar questions from a knowledge base. It uses these known answers to predict the top-3 options. The train dataset was pre-processed to remove all the prefixes and suffixes present in the prompts. The knowledge base was then build from the pre-processed training dataset. Initially, a RAG system was built using the Wikipedia dataset and a zero-shot classification model. But it did not perform well for the given dataset and was hence removed.

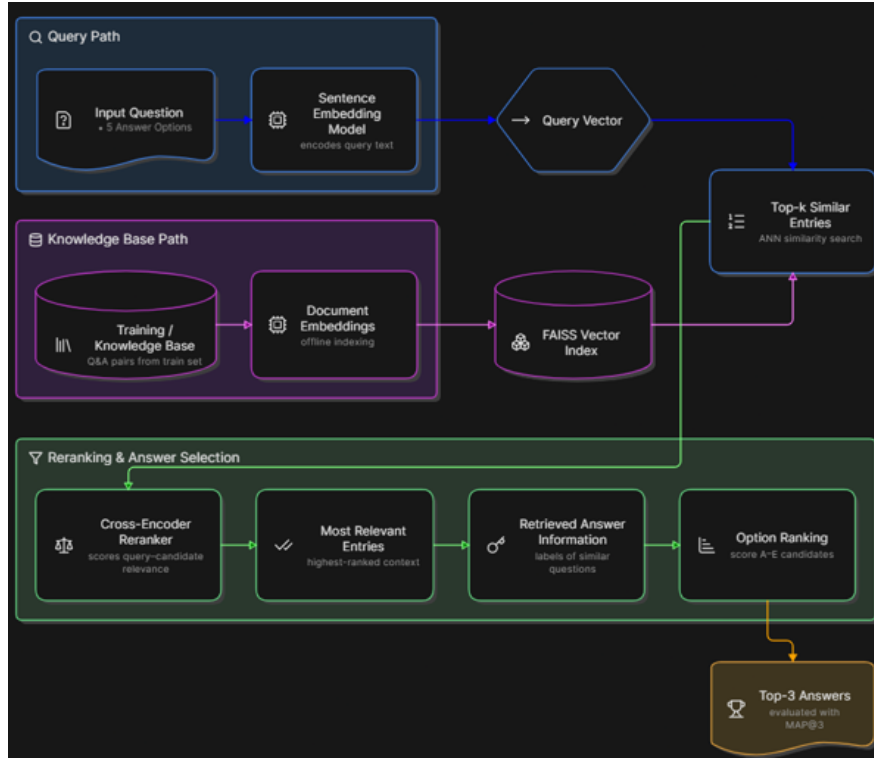


Figure 7: Arch. Diagram for Retrieval-based Approach

For each test question, the question and its five options are preprocessed using the same text-normalization pipeline used during training. The resulting query is converted into a dense vector representation using a sentence-transformer embedding model. FAISS index is then used to efficiently retrieve the most similar entries from the knowledge base. The retrieved entries provide evidence about the likely answer to the current question.

In the train-data retrieval setting, the correct answer associated with the most similar retrieved question is used to determine the predicted option. For the more complete RAG pipeline, the retrieved context is passed to a reranking/generation stage to determine the most likely answers among options A–E.

Salient Points of this approach include semantic retrieval which allows similar questions to be matched, and FAISS indexing which provides efficient similarity search. The retrieval approach was chosen over a traditional RAG system since it was observed that that the competition dataset contained questions that were already in the train dataset. So instead of using an external LLM or SLM to predict the answer for every question, retrieving similar examples allows the system to use the information already present in the training corpus. The retrieval-based approach using the training dataset achieved a near-perfect validation MAP@3 of 0.9975.

V. Performance & Comparative Analysis

The following metrics were used during model evaluation:

- **MAP@3 (Primary Metric):** Evaluates how highly ranked the correct answer is. Penalizes lower ranking or not ranking at all in top-3.
- **Top-1 Accuracy:** Evaluates the model’s ability to rank the correct answer as the highest-ranked option.

- **Top-3 Accuracy:** Measures the percentage of questions for which the correct answer appears anywhere within the model's top three predictions.
- **Macro-F1 Score:** Calculates the F1 score independently for each answer class (A–E) and then takes their unweighted average.

The models were evaluated using Stratified K-Fold Cross-Validation to maintain a similar distribution of answer classes across the training and validation folds.

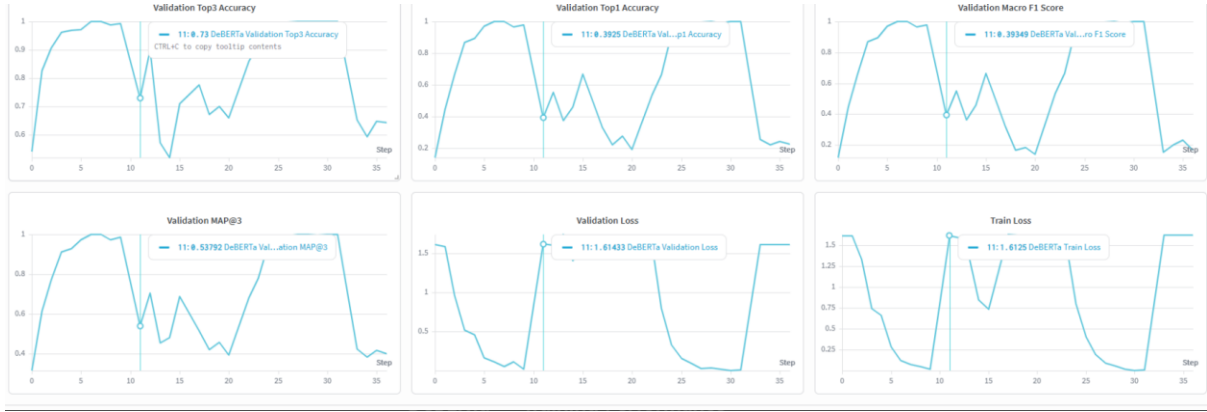


Figure 8: Metrics of DeBERTa

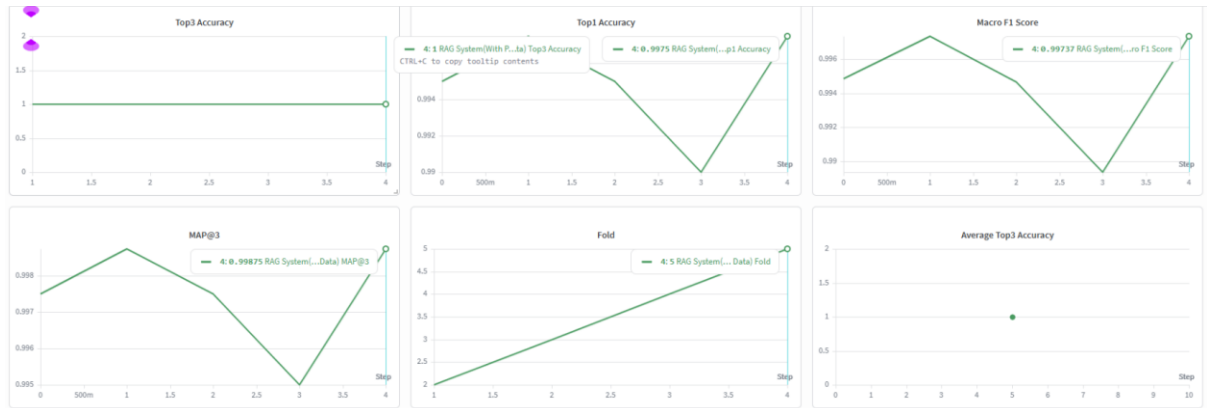


Figure 9: Metrics of Retrieval-based Approach

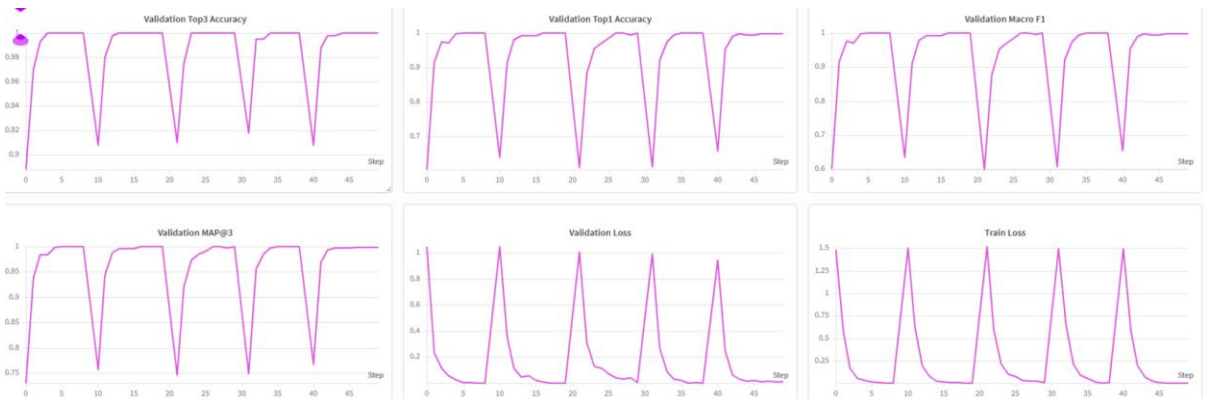


Figure 10: Metrics of BiLSTM

The cross-validation results in Figure 8 indicate that DeBERTa performs very strongly on some folds but substantially worse on others. This variability suggests that the model's performance is sensitive to the composition of the validation folds. Overall, the DeBERTa model

demonstrates strong potential for the MCQ ranking task but only the best checkpoint providing perfect MAP@3 score should be used for predicting and evaluation. The results of the retrieval-based approach in Figure 9 shows that the Top-3 Accuracy remained at 1.00. This indicates that the correct answer was included among the three retrieved/ranked candidates for essentially all validation samples. The overall Top-1 Accuracy was approximately 0.995, while the Macro F1-score was approximately 0.994 and an overall MAP@3 of 0.9975. The validation curves in Figure 10 show that the BiLSTM model learns rapidly during the initial training steps. The metrics increases to nearly 1.00, during the later portions of training. This indicates that the BiLSTM + Attention architecture is capable of effectively learning the prompts and its answers.

The results in Table 1 show that increasing the model complexity does not necessarily guarantee better score. The simple TF-IDF baseline achieved a high validation MAP@3 of 0.959, showing that the prompts and options were very similar in the dataset.

Table 1: Metrics of All Models

Model	Macro F1	Top-1 Accuracy	Top-3 Accuracy	MAP@3	Kaggle Score	Main Advantage
TF-IDF+Logistic Regression	-	0.95212	0.99505	0.97129	0.73441	Computationally efficient and Simple
Qwen2.5-7B-Instruct	-	0.81671	0.97633	0.88845	0.66625	Baseline with no training
DeBERTa	1	1	1	1	—	Strong Contextual Representation
Retrieval-based	0.99473	0.995	1	0.9975	0.75103	Exploits similar questions in knowledge base
Attention-based BiLSTM	1	1	1	1	0.7643	Task-specific learning

The fine-tuned DeBERTa model improved upon this baseline by learning contextual relationships between questions and candidate answers. Similarly, the retrieval-based approach achieved the highest validation score of approximately **0.9975**. However, its exceptional validation performance should be interpreted with caution because the retrieval system can directly exploit highly similar examples present in the training knowledge base.

The BiLSTM model provided an important contrast. Although its architecture was considerably smaller than a large pretrained Transformer, it achieved the **highest Kaggle leaderboard score of 0.7643**. This suggests that its learned representation and task-specific architecture generalized effectively to the competition's hidden test set.

The different approaches also involved different computational requirements. TF-IDF + Logistic Regression had the lowest computational cost and was suitable as a fast baseline. BiLSTM + Attention required substantially more computation during training but remained considerably lighter than large pretrained language models. DeBERTa required greater GPU memory and computational resources because of its pretrained Transformer architecture and fine-tuning process. Qwen2.5-7B had the highest computational requirements among the tested models and also encountered GPU memory limitations during experimentation. The retrieval-based system required the additional computational cost of generating embeddings,

constructing a FAISS index, performing similarity search, and, where applicable, reranking retrieved candidates.

The final BiLSTM model achieved the strongest submitted Kaggle result of **0.7643** MAP@3, achieving a leaderboard rank of **76**.

VI. Conclusion & Future Work

The key learning from this project was to thoroughly analyse the dataset before moving on to model building. The EDA helped with picking the most appropriate parameters such as max_tokens for tokenizer and helped in finding out about the data leakage in train and test dataset where 455 of the 500 test dataset questions are present in the train dataset.

The project provided experience with fine-tuning pretrained Transformer models using Hugging Face. The DeBERTa model demonstrated the effectiveness of pretrained contextual representations for multiple-choice question answering by achieving 0.98 validation score. The attention-based BiLSTM provided an important understanding of custom deep learning architectures. By combining bidirectional sequence processing with an attention mechanism, the model was able to identify the prompt and the options and predict the correct option. It achieved the best Kaggle leaderboard score of 0.7643, since it only trained on the train dataset which provided the same structure of questions in the test dataset.

Another important learning was the use of retrieval-based methods. The retrieval approach achieved a near-perfect validation MAP@3 of approximately **0.9975** when using the training dataset as a knowledge base. This demonstrated the usefulness of retrieving semantically similar examples for MCQ prediction. This project also provided practical experience in using effective tools such as wandb and hugging face in developing and evaluating multiple modelling approaches.

Several challenges were encountered during the development process. One major challenge was the computational requirement of large pretrained models. Experiments with large language models such as Qwen2.5-7B required substantial GPU memory and were computationally demanding. Similarly, there were also challenges in implementing the DeBERTa model with memory allocation issues with GPU. Since the batch size was higher, the memory ran out quickly making the training stop. It was then reduced to 2 which helped to run the model smoothly.

Another challenge was handling the data leakage and low-quality data. All the test prompts can be sourced back to the train dataset but it did not provide high MAP@3 Leaderboard score. Model implementation and checkpoint management also required careful attention, particularly when modifying the BiLSTM architecture during experimentation. Changes to layers such as the attention mechanism, normalization layer, and classifier required the model architecture used during inference to remain consistent with the architecture used when the checkpoint was trained.

Future improvements include ensembling the predictions from the BiLSTM and DeBERTa or RoBERTa models using weighted probability averaging or rank-based ensembling. The retrieval system could be improved by using a more diverse knowledge base rather than relying on the train dataset alone. Other pretrained models such as RoBERTa, or other encoder-based architectures could be fine-tuned using task-specific multiple-choice objectives. Parameter-efficient fine-tuning methods such as LoRA and hyper parameter tuning using Optuna could also be explored to reduce computational requirements. Constructing

validation splits that minimize semantically overlapping questions could be utilised between training and validation sets. This would provide a more realistic estimate of performance on unseen competition data.

VII. References

- [1] Vaswani, Ashish & Shazeer, Noam & Parmar, Niki & Uszkoreit, Jakob & Jones, Llion & Gomez, Aidan & Kaiser, Lukasz & Polosukhin, Illia. (2017). Attention Is All You Need.
- [2] Zhou, Peng & Shi, Wei & Tian, Jun & Qi, Zhenyu & Li, Bingchen & Hao, Hongwei & Xu, Bo. (2016). Attention-Based Bidirectional Long Short-Term Memory Networks for Relation Classification. 207-212.
- [3] Devlin, Jacob & Chang, Ming-Wei & Lee, Kenton & Toutanova, Kristina. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.
- [4] He, Pengcheng & Gao, Jianfeng & Chen, Weizhu. (2021). DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing.
- [5] Alshammari, Ahmad. (2023). Implementation of Keyword Extraction using Term Frequency-Inverse Document Frequency (TF-IDF) in Python. International Journal of Computer Applications.
- [6] Hugging Face, “Transformers: State-of-the-Art Machine Learning for PyTorch, TensorFlow, and JAX.”. Hugging Face documentation and model repository.
- [7] PyTorch: An Open-Source Machine Learning Framework. PyTorch documentation.
- [8] Weights & Biases, “Weights & Biases (W&B): Experiment Tracking and Machine Learning Platform,” used for monitoring training loss, validation MAP@3, Top-1 accuracy, Top-3 accuracy, F1-score, and learning-rate progression.